31. Python: Pandas Series

Turning raw data into meaningful insights.

Previous Sections:

</> 30.1. Python: NumPy

</> 30.2. Python: SciPy

</> 30.3. Python: SymPy

Contents:

So far, most of the data we have worked with has been relatively small. A few variables. A handful of values. Maybe a list, a dictionary, or a simple file.

But real-world data rarely looks like that. Companies collect millions of customer records. Scientists analyze massive experimental datasets. Financial institutions process transactions every second. And modern Machine Learning systems are trained on datasets containing thousands—or even millions—of rows.

At some point, basic Python data structures start to become inconvenient. We need something designed specifically for working with data. This is where **Pandas** comes in.

Pandas is one of the most important libraries in the entire Python ecosystem. It provides powerful tools for loading, organizing, cleaning, transforming, and analyzing structured data efficiently. Whether the data comes from a CSV file, an Excel spreadsheet, a database, or even an online source, Pandas gives us a consistent way to work with it.

In this chapter, we are only scratching the surface. When we later study **Data Analytics**, **Data Science**, and **Machine Learning**, we will revisit Pandas in much greater depth. Entire courses can be built around this library alone.

For now, our goal is simple: To become comfortable with the core data structures and operations that make Pandas one of the most widely used tools in modern computing.

By the end of this chapter, you will be able to load datasets, inspect information, manipulate data, perform statistical analysis, and begin exploring data in a way that would be extremely difficult using standard Python alone.

1. Introduction to Pandas

Before we start writing code, let's first understand what Pandas actually is and why it has become such a fundamental tool in data analysis.

1.1. What is Pandas?

Pandas is a fast, powerful, and open-source Python library designed for working with structured and tabular data.

The name *Pandas* comes from two ideas: **Panel Data** (multidimensional statistical data) and **Python Data Analysis.** Its primary purpose is to make data manipulation and analysis simple, efficient, and intuitive.

Using Pandas, we can:

- Read data from various sources
- Organize information into structured formats
- Clean and transform datasets
- Perform calculations and statistical analysis
- Prepare data for visualization and Machine Learning

In practice, Pandas acts like a highly intelligent spreadsheet system that can be controlled entirely through Python code.

1.2. Why is Pandas Used?

Imagine trying to analyze a dataset containing hundreds of thousands of rows using only lists and loops. It quickly becomes slow, repetitive, and difficult to manage.

Pandas solves this problem by providing specialized data structures and optimized functions specifically designed for handling data.

Some of its major advantages include:

- **Efficient Data Handling:** Pandas is built to work with large datasets efficiently. Whether the data contains hundreds of rows or millions, Pandas provides tools that make retrieval, filtering, sorting, and processing significantly easier.
- **Powerful Data Cleaning Tools**: Real-world datasets are rarely perfect. Missing values, duplicated records, inconsistent formats, and incorrect

entries are common problems. Pandas provides built-in functions to:

  - Remove duplicates
  - Handle missing values
  - Replace incorrect data
  - Normalize values
  - Encode categorical information

- **Rich Analytical Capabilities:** Pandas makes statistical analysis straightforward. Common operations include: Sum; Mean; Median; Mode; Variance; Standard deviation; Quantiles; Correlation analysis; and many more.
- Excellent Ecosystem Integration: Pandas works seamlessly with many other Python libraries:

  - **NumPy** for numerical computation
  - **Matplotlib** for visualization
  - **Seaborn** for advanced plotting
  - **Scikit-learn** for Machine Learning

  Because of this integration, Pandas often serves as the central data-processing layer in many Python projects.

1.3. When Should You Use Pandas?

Pandas is useful whenever your program needs to work with structured data. Common use cases include:

- **Loading Data:** Reading data from lists, dictionaries, CSV files, Excel spreadsheets, SQL databases, web APIs
- **Cleaning Data:** Preparing data before analysis by removing invalid records, handling missing values, converting formats, normalizing values, encoding categories
- **Exploring Data:** Understanding datasets through filtering, sorting, grouping, aggregation, data summarization

- **Statistical Analysis:** Performing operations such as mean, median, mode, variance, standard deviation, percentiles, correlations
- **Machine Learning Preparation:** Before training a Machine Learning model, data often needs to be cleaned, transformed, encoded, normalized. Pandas is one of the primary tools used for these tasks.

1.4. Importing Pandas

Before using Pandas, we need to import it.

```
import pandas as pd
```

The alias `pd` is the standard convention used by virtually every Python programmer working with Pandas.

This allows us to write:

```
pd.Series(...)
pd.DataFrame(...)
pd.read_csv(...)
```

instead of repeatedly typing:

```
pandas.Series(...)
pandas.DataFrame(...)
pandas.read_csv(...)
```

And now that we understand what Pandas is and why it is so useful, let's begin with its first and most fundamental data structure:

2. Pandas Series

The first and most fundamental data structure in Pandas is the **Series**.

A **Series** is a one-dimensional labeled array capable of storing a collection of values. Unlike a normal Python list, every value in a Series is associated with an **index label**, creating an index-value relationship similar to a key-value pair in a dictionary.

Think of a Series as a single column of data with row labels attached to it. For example:

Index	Value
0	Alice
1	Bob
2	Charlie

In fact, whenever you select a single column from a DataFrame, Pandas automatically returns a Series.

A Series can be created from Python Lists; NumPy Arrays; Dictionaries

2.1. Structure of a Series

Every Series contains two main components:

- **Index:** The labels used to identify each element.
- **Value (Data):** The actual information stored in the Series.

For example:

```
import pandas as pd

data = ["Alice", "Bob", "Charlie"]

s = pd.Series(data)
print(s)
```

```
0     Alice
1       Bob
2   Charlie
dtype: object
```

Here: `0, 1, 2` are the indexes; `Alice, Bob, Charlie` are the values

2.2. Creating a Series

- Creating a Series from a List

```
import pandas as pd

data = [10, 20, 30, 40]

s = pd.Series(data)
print(s)
```

```
0   10
1   20
2   30
3   40
dtype: int64
```

- Giving the Series a Name

```
import pandas as pd

data = [10, 20, 30, 40]

s = pd.Series(data, name="Scores")
print(s)
```

```
0   10
1   20
2   30
3   40
Name: Scores, dtype: int64
```

- Custom Index Labels

```python
import pandas as pd

data = [85, 90, 95]

s = pd.Series(data, index=["Alice", "Bob", "Charlie"])
print(s)
```

```
Alice     85
Bob       90
Charlie   95
dtype: int64
```

- Creating a Series from a Dictionary

When using a dictionary, the keys automatically become indexes.

```python
import pandas as pd

data = {"Alice": 85, "Bob": 90, "Charlie": 95}

s = pd.Series(data)
print(s)
```

```
Alice     85
Bob       90
Charlie   95
dtype: int64
```

2.3. Renaming a Series

The `rename()` method can change index labels or the Series name.

```python
import pandas as pd

data = [100, 90, 80]

s = pd.Series(data, index=["A", "B", "C"])
s = s.rename(index={"A": "Alice"})
print(s)
```

```
Alice   100
B        90
C        80
dtype: int64
```

Changing the Series name:

```python
s.name = "Exam Scores"
print(s)
```

```
Alice   100
B        90
C        80
Name: Exam Scores, dtype: int64
```

2.4. Series Size and Shape

- Number of Rows

```python
import pandas as pd

data = [10, 20, 30, 40]

s = pd.Series(data)
print(len(s))
```

```
4
```

- Total Number of Elements

```
print(s.size)
```

4

- Shape of a Series

```
print(s.shape)
```

(4,)

Notice the comma. A Series only has one dimension, so its shape is represented as: `(rows,)`

2.5. Traversing Through a Series

Use `items()` to iterate through indexes and values.

```
import pandas as pd

data = [85, 90, 95]
s = pd.Series(data, index=["Alice", "Bob", "Charlie"])

for key, value in s.items():
    print(key, value)
```

Alice 85
Bob 90
Charlie 95

2.6. Filtering a Series

Filtering works similarly to NumPy arrays.

```
import pandas as pd

data = [60, 75, 90, 55, 88]

s = pd.Series(data)
print(s[s >= 80])
```

```
2   90
4   88
dtype: int64
```

2.7. Insertion and Deletion

- Insert a New Value

```
import pandas as pd

data = [10, 20, 30]

s = pd.Series(data)
s[3] = 40
print(s)
```

```
0   10
1   20
2   30
3   40
dtype: int64
```

- Delete a Value

```
s = s.drop(1)
print(s)
```

```
0   10
2   30
3   40
dtype: int64
```

2.8. Keys and Values

- Retrieve Index Labels

```
import pandas as pd

data = [85, 90, 95]

s = pd.Series(data, index=["Alice", "Bob", "Charlie"])
print(s.keys())
```

```
Index(['Alice', 'Bob', 'Charlie'], dtype='object')
```

- Retrieve Values

```
print(s.values)
```

```
[85 90 95]
```

- Retrieve Value from Key

```
print(s["Bob"])
```

```
90
```

- Retrieve Key from Value

```
print(s[s == 95].index)
```

```
Index(['Charlie'], dtype='object')
```

2.9. Indexing and Slicing

- Position-Based Indexing

```
import pandas as pd

data = [10, 20, 30, 40, 50]

s = pd.Series(data)
print(s.iloc[2])
```

```
30
```

- Access Index Label by Position

```
print(s.index[2])
```

```
2
```

- Slicing

```
print(s.iloc[1:4])
```

```
1   20
2   30
3   40
dtype: int64
```

2.10. Unique Values

```
import pandas as pd

data = [10, 20, 20, 30, 30, 30]

s = pd.Series(data)
print(s.unique())
```

[10 20 30]

The `unique()` method returns the distinct values found in the Series.

2.11. Sorting a Series

- Sort by Values

```
import pandas as pd

data = [50, 20, 80, 10]

s = pd.Series(data)
print(s.sort_values())
```

```
3   10
1   20
0   50
2   80
dtype: int64
```

- Sort by Index

```
print(s.sort_index())
```

```
0   50
1   20
2   80
3   10
dtype: int64
```

2.12. Converting a Series to a DataFrame

A Series can be converted into a DataFrame using `to_frame()`.

```
import pandas as pd

data = [100, 90, 80]

s = pd.Series(data, name="Scores")

df = s.to_frame()
print(df)
```

```
   Scores
0     100
1      90
2      80
```

Notice that the Series becomes a single-column DataFrame.

2.13. From Series to DataFrame and Back

A dictionary containing list values creates a DataFrame instead of a Series.

```
import pandas as pd

data = {"Name": ["Alice", "Bob", "Charlie"],
        "Score": [85, 90, 95]}

df = pd.DataFrame(data)
print(df)
```

```
      Name  Score
0    Alice     85
1      Bob     90
2  Charlie     95
```

Retrieving a single column from a DataFrame produces a Series.

```
scores = df["Score"]

print(type(scores))
print(scores)
```

```
<class 'pandas.core.series.Series'>

0    85
1    90
2    95
Name: Score, dtype: int64
```

This relationship is important:

- A **Series** can become a **DataFrame** using `to_frame()`.
- A single column selected from a **DataFrame** becomes a **Series**.

You can think of a Series as the building block of a DataFrame. A DataFrame is essentially multiple Series combined together into a table.

Summary

In this section, we explored the **Pandas Series**, the most fundamental data structure in the Pandas library.

A Series is a **one-dimensional labeled array** that stores data as index-value pairs, making it much more powerful than a simple Python list. We learned how to create Series objects from lists and dictionaries, assign custom indexes and names, rename labels, and inspect their size and shape.

We also covered many common operations, including:

- Traversing through a Series with `items()`
- Filtering values using conditions
- Inserting and deleting elements
- Retrieving indexes and values
- Indexing and slicing data

- Finding unique values
- Sorting by values or indexes
- Converting a Series into a DataFrame

Most importantly, we discovered the close relationship between **Series** and **DataFrames**. A Series can be converted into a DataFrame, while selecting a single column from a DataFrame automatically produces a Series. You can think of a Series as a single column of data with labels attached to it.

But real-world datasets rarely contain just one column. Most data consists of multiple related columns organized into tables. This naturally leads us to the next and most important Pandas structure: The **DataFrame**—a collection of Series working together to represent complete datasets.

References

https://pandas.pydata.org/docs/reference/api/pandas.Series.html

https://www.geeksforgeeks.org/pandas/python-pandas-series/

Next Section:

</> 31.1. Python: Pandas DataFrame